

Automated Testing, QA & CI/CD Deployment Strategy

Project: HireSense (AI-Powered Applicant Tracking System)

Document Version: 1.0.0

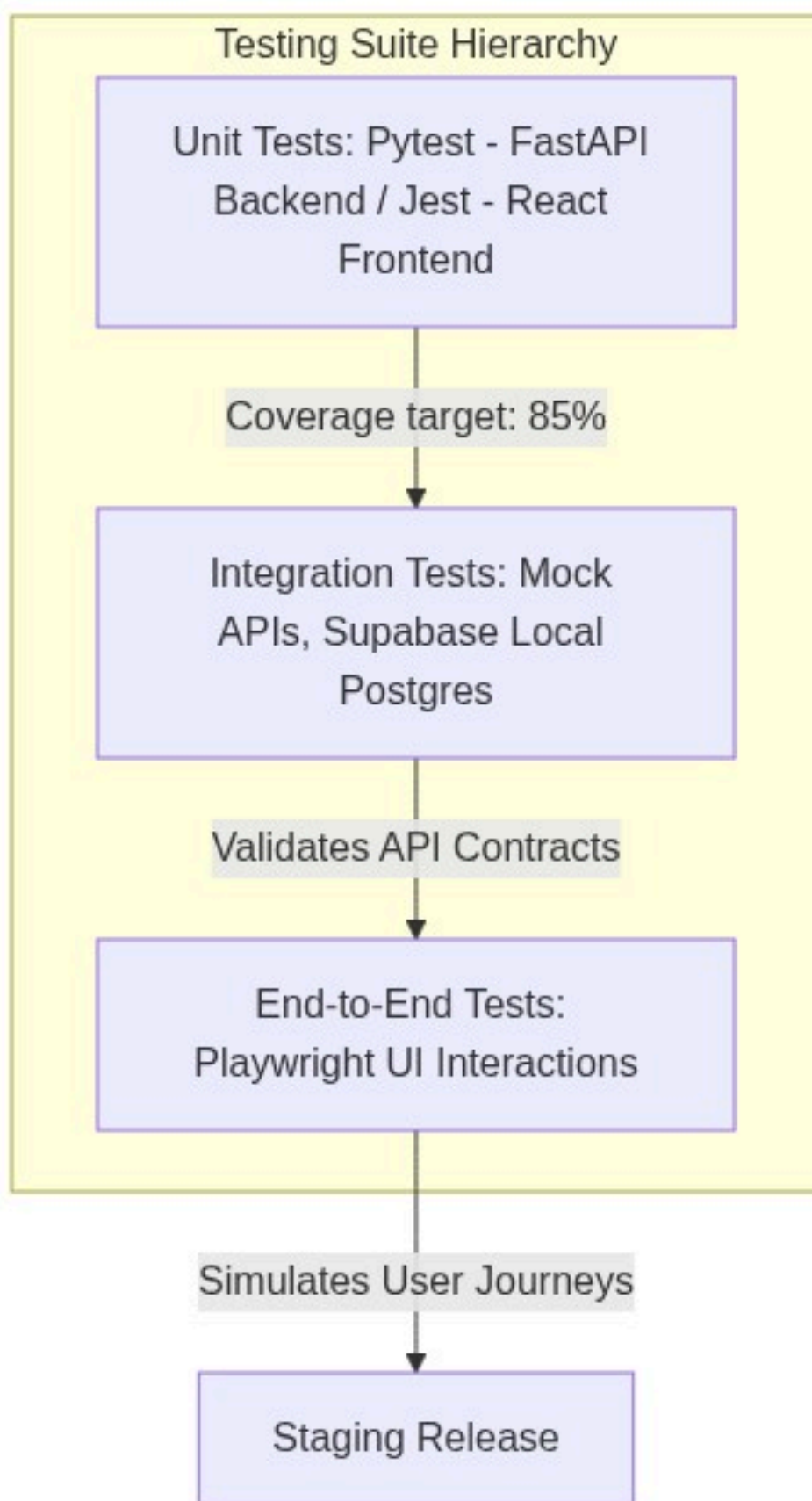
Author: Antigravity Principal QA & DevOps Engineer

Status: Active

Target Environment: GitHub Actions, Pytest, Playwright, Vercel & AWS Deployments

1. Multi-Tier Testing Strategy

To ensure zero-downtime releases and maintain high system reliability, HireSense implements a three-tier automated testing hierarchy.



1.1 Testing Tier Definitions

- Unit Testing (FastAPI & React):

- Backend: Evaluates isolated Python functions, Pydantic model validations, and security RLS helper calculations.
 - Frontend: Evaluates React hooks, utility functions, and component rendering states.
 - Integration Testing:
 - Validates interaction boundaries between the FastAPI router gateway, Supabase local PostgreSQL instances, and the Redis cache queue.
 - Leverages mock interfaces for external LLM API endpoints to prevent excessive API costs during testing cycles.
 - End-to-End (E2E) Testing:
 - Automates complete user journeys inside headless web browsers (Chrome, Firefox, Safari) using Playwright.
 - Validates complex components such as file drag-and-drop actions, candidate profile expansions, and real-time status transitions on the Kanban board.
-

2. Backend Pytest Specification & Mocks

All backend tests are run using the Pytest library in Python, leveraging async database fixtures and modular mock setups.

2.1 Pytest Client & Database Fixtures (`confest.py`)

This script configures a headless client and registers isolated transactional database contexts:

```
import pytest
from httpx import AsyncClient
from typing import AsyncGenerator
from fastapi import FastAPI
from backend_v3.main import app
from backend_v3.core.database import get_db

@pytest.fixture(scope="session")
def anyio_backend() -> str:
    return "asyncio"

@pytest.fixture
async def client() -> AsyncGenerator[AsyncClient, None]:
    # Set up HTTPX client targeting local mock server instance
    async with AsyncClient(app=app, base_url="http://test") as ac:
        yield ac
```

2.2 Endpoint Verification Spec (`test\_resumes.py`)

Validates that uploading a PDF correctly triggers the async resume processing queue:

```
import pytest
from unittest.mock import AsyncMock, patch

@pytest.mark.anyio
@patch("backend_v3.services.resume_service.enqueue_resume_processing")
async def test_upload_resume_success(mock_enqueue, client):
```

```
# Setup mock return value for Redis task enqueue
mock_enqueue.return_value = "task_987654"

# Define file payload mock
files = {"file": ("test_resume.pdf", b"%PDF-1.4 dummy contents", "application/pdf")}
headers = {"Authorization": "Bearer mock_valid_jwt_token"}

response = await client.post("/api/v1/resumes/upload", files=files, headers=headers)

assert response.status_code == 202
data = response.json()
assert data["status"] == "enqueued"
assert data["task_id"] == "task_987654"
mock_enqueue.assert_called_once()
```

3. Playwright E2E Automation Setup

Playwright automates browser actions to verify complex recruiter dashboards and candidate submission steps.

3.1 Kanban Board E2E Test Suite (`kanban.spec.ts`)

This Node.js Playwright test verifies that dragging a candidate's card between columns updates the candidate's state and launches background database transactions:

```
import { test, expect } from '@playwright/test';

test.describe('Recruiter Kanban Dashboard', () => {
  test.beforeEach(async ({ page }) => {
    // 1. Authenticate and seed active dashboard state
    await page.goto('/login');
    await page.fill('#email', 'recruiter@hiresense.ai');
    await page.fill('#password', 'SecurePassword123');
    await page.click('button[type="submit"]');
    await expect(page).toHaveURL('/dashboard');
  });

  test('Should drag-and-drop a candidate profile to next stage', async ({ page }) => {
    // Locate the first candidate card in 'AI Screened'
    const card = page.locator('[data-testid="candidate-card-john-doe"]').first();
    // Locate the destination Kanban column: 'Interviewing'
    const destinationColumn = page.locator('[data-testid="column-interviewing"]');

    // Drag the card and drop it in the target column
    await card.dragTo(destinationColumn);

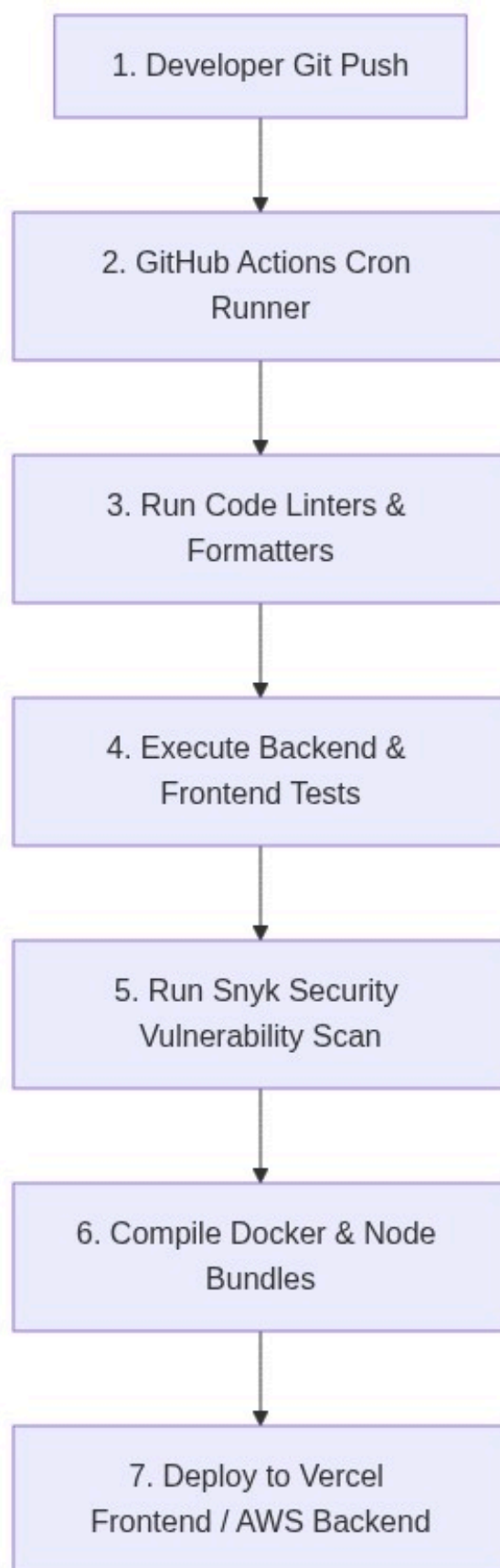
    // Verify visual update and state toast confirmation
    const successToast = page.locator('.toast-success');
    await expect(successToast).toBeVisible();
    await expect(successToast).toContainText('Candidate status updated to: Interviewing');

    // Confirm candidate exists inside the target column DOM structure
    await expect(destinationColumn.locator('[data-testid="candidate-card-john-doe"]')).toBeVisible();
  });
});
```

```
});  
});
```

4. Automated CI/CD Pipelines (GitHub Actions)

Every pull request or commit to the main branch triggers the CI/CD pipeline, automatically validating code structure, running unit/integration suites, and deploying safely.



4.1 Production Pipeline Configuration (`.github/workflows/production\_pipeline.yml`)